

UNIVERSITY OF THE ARMED FORCES - ESPE

Computer Science Department

Object Oriented Programming

36863

Homework 09 ChickenFarmAppCreateandRead

Tiban Camalle Andy Mathew

October 21, 202

Within the Java implementation of ChickenFarmApp, the newly created InputHandler.java class prompts the user via console for chicken details, validates ID uniqueness, and returns a new Chicken object instance.

```java
public static Chicken createChickenFromConsole(Chicken[] currentChickens) {
    Scanner scanner = new Scanner(System.in);

    System.out.println("\n=== New Chicken Register ===");

    int id;
    while (true) {
        System.out.print("Enter the ID: ");
        id = scanner.nextInt();
        scanner.nextLine();
        if (isIdTaken(id, currentChickens)) {
            System.out.println("[!] Error: That ID is already registered. Please enter another one.");
        } else {
            break;
        }
    }
    System.out.print("Enter the name: ");
    String name = scanner.nextLine();

    System.out.print("Enter the color: ");
    String color = scanner.nextLine();

    System.out.print("Enter the year of birth: ");
    int yearOfBirth= scanner.nextInt();

    System.out.print("Enter the month of birth: ");
    int monthOfBirth= scanner.nextInt();

    System.out.print("Enter the day of birth: ");
    int dayOfBirth= scanner.nextInt();

    System.out.print("The chicken is molting? (true or false): ");
    boolean molting = scanner.nextBoolean();


    return new Chicken(id, name, color, new Date(yearOfBirth-1900,monthOfBirth,dayOfBirth), molting);
}
```

Within the Java implementation of ChickenFarmApp, the newly created CsvManager.java class exports the complete array of registered chicken objects into a formatted CSV file.

Output × Egg.java × ChickenFarmApp.java × CsvManager.java × InputHandler.java × C

Source History

```java
package ec.edu.espe.chickenfarm.controller;

import ec.edu.espe.chickenfarm.model.Chicken;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

/**
 *
 * @author Andy Tiban, IV Studio, @ESPE
 */
public class CsvManager {
    private static final String FILE_PATH = "chickens.csv";

    public static void saveChickens(Chicken[] chickens) {

        try (PrintWriter writer = new PrintWriter(new FileWriter(FILE_PATH))) {


            writer.println("id,name,color,bornOnDate,ageInYears,molting");


            if (chickens != null) {
                for (Chicken c : chickens) {
                    if (c != null) {

                        writer.println(c.getId() + "," +
                                       c.getName() + "," +
                                       c.getColor() + "," +
                                       c.getBornOnDate() + "," +
                                       c.getAgeInYears() + "," +
                                       c.isMolting());
                    }
                }
            }
            System.out.println("[OK] Data successfully saved in: " + FILE_PATH);

        } catch (IOException e) {
            System.out.println(" Could not save the CSV file: " + e.getMessage());
        }
    }
}
```

Within the Java implementation of ChickenFarmApp, the newly created JsonManager.java class exports the complete array of registered chicken objects into a formatted JSON file.

Output × Egg.java × ChickenFarmApp.java × CsvManager.java × InputHandler.java × Chicken.java × JsonManager.java ×

Source History

```java
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import ec.edu.espe.chickenfarm.model.Chicken;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

/**
 *
 * @author Andy Tiban, IV Studio, @ESPE
 */
public class JsonManager {

    private static final String FILE_PATH = "chickens.json";

    public static void saveChickens(Chicken[] chickens) {
        Gson gson = new GsonBuilder().setPrettyPrinting().create();
        try (FileWriter writer = new FileWriter(FILE_PATH)) {
            gson.toJson(chickens, writer);
            System.out.println("\n[OK] Data successfully saved in: " + FILE_PATH);
        } catch (IOException e) {
            System.out.println("\n[ERROR] The file could not be saved: " + e.getMessage());
        }
    }


    public static Chicken[] loadChickens() {
        Gson gson = new Gson();
        try (FileReader reader = new FileReader(FILE_PATH)) {
            return gson.fromJson(reader, Chicken[].class);
        } catch (IOException e) {
            System.out.println("\n[AVISO] No previous file was found, or an error occurred: " + e.getMessage());
            return new Chicken[10];
        }
    }

```

Final Result

```
=== Farmer info ===
ChickenFarmer{chickenFarmerId=1, name=Andy, coops=[null, null]}

=== New Chicken Register ===
How many chickens do you want to add?: 1

--- Entering new chicken data N.1 ---

=== New Chicken Register ===
Enter the ID: 4
[!] Error: That ID is already registered. Please enter another one.
Enter the ID: 5
Enter the name: Richard
Enter the color: brown
Enter the year of birth: 2025
Enter the month of birth: 1
Enter the day of birth: 1
The chicken is molting? (true or false): false

[OK] Data successfully saved in: chickens.json
[OK] Data successfully saved in: chickens.csv

=== Reading chickens from the archive ===
Chicken{id=3, name=pedro, color=black, bornOnDate=Sat Nov 21 00:00:00 GMT-05:00 2020, ageInYears=5, molting=true}
Chicken{id=1, name=harold, color=brown, bornOnDate=Sat Nov 21 00:00:00 GMT-05:00 2020, ageInYears=5, molting=false}
Chicken{id=2, name=pepe, color=black, bornOnDate=Thu Nov 14 00:00:00 GMT-05:00 2019, ageInYears=6, molting=true}
Chicken{id=3, name=santiago, color=black  and white, bornOnDate=Mon Nov 01 00:00:00 GMT-05:00 2021, ageInYears=4, molting=false}
Chicken{id=4, name=Charlana, color=brown and black, bornOnDate=Sun Feb 01 00:00:00 GMT-05:00 2026, ageInYears=0, molting=false}
Chicken{id=5, name=Richard, color=brown, bornOnDate=Sat Feb 01 00:00:00 GMT-05:00 2025, ageInYears=1, molting=false}
BUILD SUCCESSFUL (total time: 24 seconds)
```

Within the Java implementation of ChickenFarmApp, this main workflow loads existing records via JsonManager, appends newly registered chickens captured by InputHandler, and completely overwrites both the JSON and CSV persistent files with the updated collection.

```
        Chicken[] existingChickens = JsonManager.loadChickens();


        int existingCount = 0;
        if (existingChickens != null) {
            for (Chicken c : existingChickens) {
                if (c != null) existingCount++;
            }
        }

        System.out.println("\n=== New Chicken Register ===");

        System.out.print("How many chickens do you want to add?: ");
        int numberOfChickensToAdd = scanner.nextInt();
        scanner.nextLine();


        Chicken[] totalChickens = new Chicken[existingCount + numberOfChickensToAdd];

        int currentIndex = 0;


        if (existingChickens != null) {
            for (Chicken c : existingChickens) {
                if (c != null) {
                    totalChickens[currentIndex] = c;
                    currentIndex++;
                }
            }
        }

        for(int i = 0; i < numberOfChickensToAdd; i++) {
            System.out.println("\n--- Entering new chicken data N." + (i + 1) + " ---");
            totalChickens[currentIndex] = InputHandler.createChickenFromConsole(totalChickens);
            currentIndex++;
        }


        JsonManager.saveChickens(totalChickens);
        CsvManager.saveChickens(totalChickens);
```

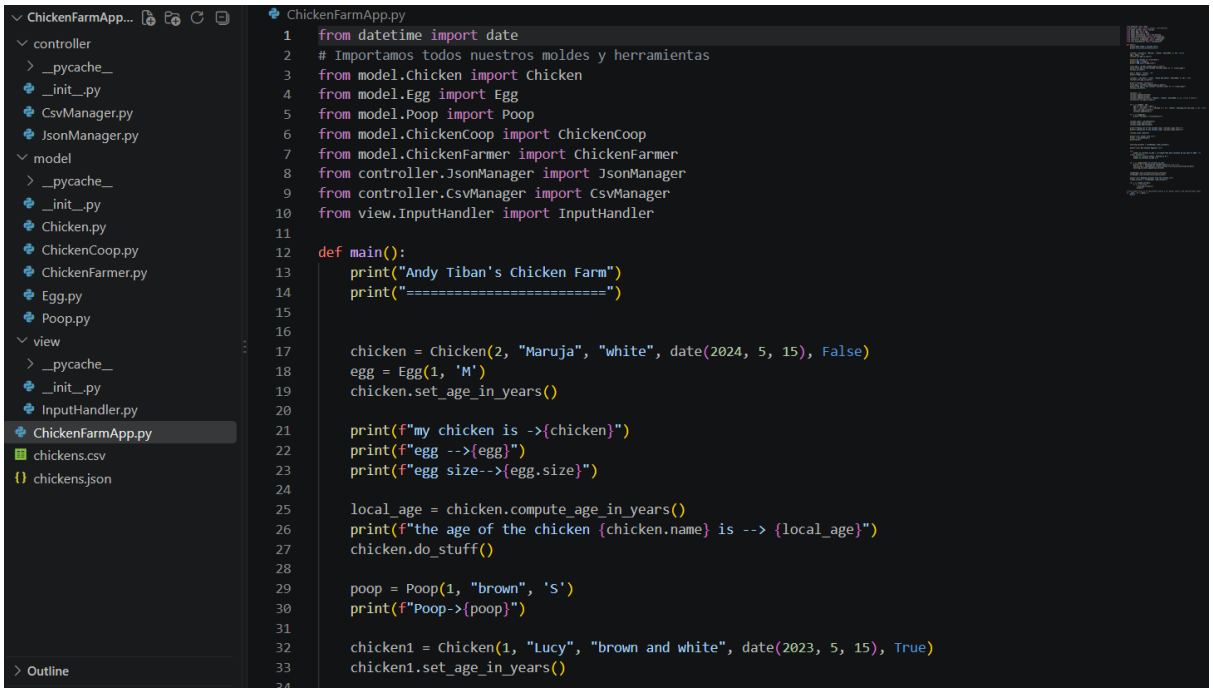
```
        System.out.println("\n=== Reading chickens from the archive ===");
        Chicken[] loadedChickens = JsonManager.loadChickens();

        for (Chicken c : loadedChickens) {
            if (c != null) {
                c.setAgeInYears();
                System.out.println(c);
            }
        }

    }
}
```

ChickenFarmApp in other programming languages

Python



Javascript

